

ResuFlow

Software Engineering Concepts — Applied Analysis

FAST NUCES Lahore · Software Architecture Course · Phase 3

1. Architectural Styles

Layering — Primary Architecture

Classification: Primary Architectural Style

The entire system is a 3-tier layered architecture:

- Presentation Layer: frontend/ (React dashboard) + portfolio-templates/ (Next.js renderer)
- Business Logic Layer: backend/routes/, backend/services/ (upload, deploy, CV extraction, validation)
- Data Access Layer: backend/models/ (UserProfile, PortfolioDeployment via Mongoose)
- Database Layer: MongoDB Atlas

Why: Each layer only communicates with the layer directly below it. The frontend never touches MongoDB directly — it goes through the API. This gives clean separation so MongoDB can be swapped for a different DB, or Gemini replaced with another AI, without touching the frontend.

Client-Server — Primary Communication Model

Classification: Primary Communication Model

Every interaction between frontend and backend follows this model:

- Clients: React (UploadForm, DeploymentScreen) and Next.js ([username]/page.tsx)
- Server: Express in backend/index.js serving /api/* routes
- Protocol: HTTP request/reply (POST, GET)

Why: The frontend initiates all requests; the backend reacts. Classic asymmetry — the server is unaware of which client is calling. This is why CORS is enabled and the backend URL is stored in .env on the frontend side.

Pipe and Filter — CV Extraction Pipeline

Classification: Data Processing Pattern

The CV processing chain in backend/services/ follows this pipeline:

```
File Upload → fileParser.js → cvExtractor.js → schemaValidator.js → MongoDB
(raw bytes)   (PDF/DOCX→text) (text→JSON via Gemini AI) (validated JSON) (stored)
```

Why: Each service is a stateless filter that transforms data and passes it forward. fileParser doesn't know about Gemini; cvExtractor doesn't know about MongoDB. Adding a new file format (e.g., .txt) only requires changing fileParser.js — nothing else.

Repository — Centralized Data Store

Classification: Data Management Style

MongoDB serves as the shared repository accessed by multiple subsystems:

- routes/upload.js writes extracted CV data → users collection
- routes/deploy.js writes portfolio config → portfoliodeployments collection
- routes/deploy.js reads profile for QR code + status check
- portfolio-templates/[username]/page.tsx reads via GET /api/profile/:username

Why: All three parts of the system (dashboard, backend, portfolio renderer) share the same data through one central store. The in-memory Map fallback ensures graceful degradation if the repository goes down.

Publish-Subscribe — Architectural Opportunity

The current multi-step workflow (Upload → Extract → Edit → Deploy) is sequential. However, the natural evolution would be event-driven notifications — for example, a 'portfolio deployed' event could trigger email, webhook, and analytics subscribers independently.

Why noted: The architecture already isolates concerns sufficiently that pub-sub could be layered on without restructuring. The deploy endpoint could publish events that multiple subscribers react to independently.

2. Design Principles

Modularity / Separation of Concerns

Every layer is split by a single responsibility:

- backend/: config/ (DB connection) | models/ (data schema) | routes/ (HTTP handling) | services/ (business logic)
- frontend/src/components/: One file per screen — LandingPage, UploadForm, ExtractionScreen, PreviewDashboard, DeploymentScreen
- portfolio-templates/src/components/: Each template has its own folder with isolated sub-components (Hero, Nav, Skills, Work, Contact)

Why: When the QR code library changes, only routes/deploy.js is touched. When a template's styling breaks, only that template's components/ folder is examined. Nothing bleeds across boundaries.

Information Hiding

Internal complexity is hidden behind clean interfaces:

- cvExtractor.js hides all Gemini API complexity (prompt engineering, retry logic, JSON parsing) — routes just call extractCV(text) and receive structured JSON
- fileParser.js hides whether the file is PDF or DOCX — callers get plain text either way
- Mongoose models hide MongoDB query syntax from routes
- Portfolio templates hide their internal component structure from the [username]/page.tsx router

Why: Switching from Gemini to GPT-4 only requires changing cvExtractor.js. Everything that depends on it (routes, models) is shielded from that implementation decision.

Interfaces (API Contracts)

Formal contracts exist at every system boundary:

- REST API endpoints (/api/cv/upload, /api/deploy, /api/profile/:username) — the formal interfaces between frontend and backend
- portfolio-templates/src/types/userProfile.ts — TypeScript interfaces defining the data contract between backend and templates
- Service function signatures (e.g., extractCV(text: string): Promise<CVData>) — module-level interfaces

Why: The frontend doesn't need to know how extraction works internally — it only depends on the contract (POST a file, get back JSON). This is why the Next.js portfolio renderer and React dashboard can be deployed independently on different Vercel projects yet share the same data shape.

Abstraction

Complex realities are abstracted into simpler representations:

- AppError.js — abstracts all error types into one class with status codes and messages
- fileParser.js — abstracts PDF vs DOCX into a single parseFile(buffer, mimetype) call
- The CV schema in cvExtractor.js — abstracts all professional types into one unified structure (technical, domain, soft skills)
- Template config object (palette, font, sectionsVisible) — abstracts 'customize portfolio' without exposing React component internals

Why: The ExtractionScreen component doesn't know if the user uploaded a PDF or DOCX — it just receives structured JSON. That abstraction boundary is what keeps the frontend simple.

Generality

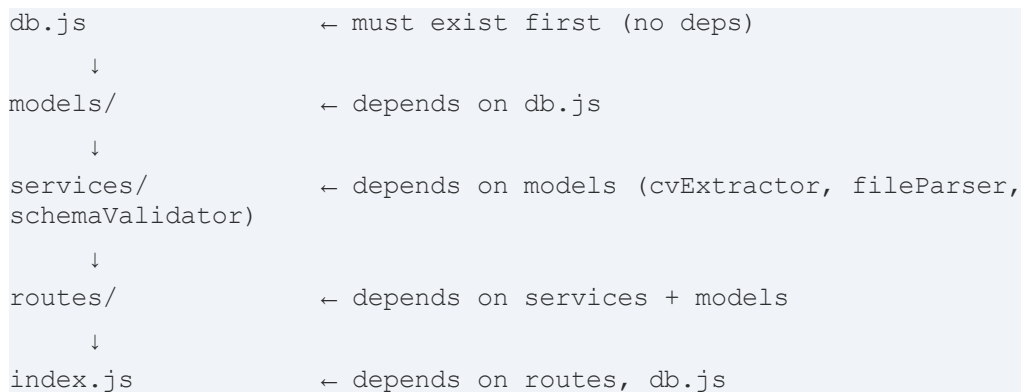
Components are designed to handle multiple cases, not hardcoded for one scenario:

- fileParser.js — generalised to handle both PDF and DOCX
- CV schema — generalised to support all professions (technical skills, domain skills, campaigns, publications, volunteering)
- Portfolio template components — generalised to accept any userProfile shape, rendering only sections that have data (sectionsVisible config)
- schemaValidator.js — generalised date normalizer that converts any date format to YYYY-MM

Why: Without generality, there would be a separate parser for PDF, another for DOCX, and a separate schema for developers vs marketers. The extras and domain skills fields exist specifically because the design was generalised beyond 'tech CV.'

Incremental Development (Fan-in / Fan-out)

The dependency hierarchy in the backend enforces a natural, verifiable build order:



- Fan-in: db.js is used by models which are used by routes — high fan-in means critical infrastructure. schemaValidator.js is called by both upload and deploy routes.
- Fan-out: index.js depends on everything — it is the entry point, appropriately high fan-out.

Why: This hierarchy means fileParser.js could be implemented and tested in complete isolation without needing the database or Gemini. The Gemini retry logic in cvExtractor.js was testable before any route existed.

Cohesion & Coupling

The system maintains high cohesion within modules and low coupling between them:

- High cohesion: cvExtractor.js does exactly one thing — call Gemini and parse CV. fileParser.js does one thing — extract text from files.
- Low coupling: Frontend never imports backend code. Templates never import frontend code. Services don't import routes. Routes don't import each other.
- Coupling via interfaces only: routes/upload.js calls extractCV() from cvExtractor.js — it depends on the function signature, not the implementation.

Why: The 75+ commit history demonstrates that templates were added iteratively without breaking upload or deploy logic — that is low coupling in practice.

3. Design by Contract (DbC)

Contracts are enforced at every system boundary:

- Preconditions enforced by Multer config: file must exist, be ≤5MB, and be of type application/pdf or .docx
- Preconditions enforced by Zod in routes/deploy.js: username, template, and profile must be valid before deploy proceeds
- Postconditions enforced by schemaValidator.js: after extraction, dates must be YYYY-MM, arrays must exist (not null)
- Invariant in rate limiter: never more than 10 deployments per hour per user
- Contract on Gemini output: cvExtractor.js validates and repairs the JSON structure before returning it

Why: Without these contracts, a malformed Gemini response could propagate all the way to the portfolio template and crash the render. The validator is the contract enforcer between the AI service and the rest of the system.

4. Component-Based Software Engineering (CBSE)

The system is built from self-contained, composable components at every layer:

- React components in `frontend/src/components/` — each is a self-contained unit with props (provided interface) hiding internal state
- Portfolio template components — `glassmorphism/Hero.tsx`, `bento/ProjectGrid.tsx` — self-contained, composable
- Third-party components — Radix UI (Accordion, Dialog, Dropdown, Select) — used as black-box components with well-defined APIs
- Backend services — `cvExtractor`, `fileParser`, `schemaValidator` — function as components defined by their interface, with implementation hidden

Why: The 9 template folders are the most direct CBSE application. Each template is an independently deployable composition of sub-components. Adding a 10th template (e.g., 'Brutalist') requires only creating a new folder and registering the route — zero changes to existing components.

5. Quality Attributes

Modifiability

The architecture localizes all changes:

- New templates: add folder to `portfolio-templates/src/components/` + register page route — no core changes
- New file format support: add case to `fileParser.js` — no route changes
- New AI provider: rewrite `cvExtractor.js` — no changes to routes, models, or frontend
- New CV fields: add to `userProfile.ts` TypeScript interface + schema prompt — templates auto-render new sections

Why it matters: The project is still evolving. The architecture ensures each addition is localized to a single module.

Reliability

Multiple mechanisms protect against external failure:

- Gemini API: retry logic with exponential backoff (up to 4 attempts, max 16s delay) in `cvExtractor.js`
- 60s timeout protection on Gemini calls, 30s on file parsing
- MongoDB graceful degradation: in-memory Map fallback if Atlas is unreachable
- `AppError` class standardizes error propagation through the stack

Why: Gemini 2.5 Flash returns 503 errors under load. Without retry logic, every CV upload during peak hours would fail. The in-memory fallback means a MongoDB outage does not take down the whole platform.

Performance

Resource constraints on Render's free tier are addressed at multiple levels:

- Rate limiting: 10 deployments/hour per user — protects the Render server from abuse
- In-memory cache: deployed portfolios cached in a Map on the backend — GET `/api/profile/:username` does not always hit MongoDB
- QR Code Level H: error correction pre-computed server-side, not client-side

Why: Render's free tier has cold starts and limited resources. The in-memory cache reduces database round-trips for the most common operation (viewing a portfolio).

Security

Security is enforced in layers across the system:

- File validation: MIME type check + 5MB size limit in Multer (prevents oversized uploads, wrong file types)
- Secrets in `.env` files (`GEMINI_API_KEY`, `MONGODB_URI` never in code or git history)
- CORS configuration: only allowed origins can call the API
- Rate limiting: prevents brute-force/spam deployments
- Zod validation: all deploy payloads validated before touching the database

Why: File uploads are a classic attack surface. Without MIME validation, a malicious executable could be uploaded. The `.env` pattern means the API key never appears in git history.

Robustness (Mutual Suspicion)

The system assumes any other component might send bad data:

- `routes/deploy.js` re-validates all data received from the frontend using Zod — even though the frontend already validated it
- `schemaValidator.js` defensively handles null/undefined fields from Gemini output, providing safe defaults
- Backend rate limiter does not trust the frontend to self-limit requests

Why: The principle of mutual suspicion — the backend treats frontend input as untrusted. This is why Zod validation exists on the backend even though the frontend has its own form validation.

Usability (Golden Rules of UI Design)

Rule 1 — User in Control

- Multi-step workflow (Upload → Extract → Preview → Deploy) — user sets the pace, never forced to skip

- ExtractionScreen — user can edit, add, or remove any AI-extracted data before deploying
- Breadcrumb navigation — user can go back at any step

Rule 2 — Reduce Memory Load

- localStorage persistence (resuflow_profile) — data survives page refresh, user doesn't re-enter anything
- Pre-filled extraction form — AI fills in everything, user only corrects mistakes
- Step labels in breadcrumb — user always knows where they are in the process

Rule 3 — Consistency

- Navbar.tsx appears on every screen (same component, same behavior)
- Radix UI components used throughout — consistent interaction patterns (dropdowns, dialogs all behave the same)
- Consistent Tailwind design tokens across all 3 parts of the system

6. Requirements Engineering

Functional Requirements

What the system does:

- CV upload (PDF/DOCX) → extraction → editing → template selection → deployment
- QR code generation for each portfolio
- Email signature export
- Custom portfolio URL per username

Non-Functional Requirements

How well the system does it:

- Performance: Respond within 60s for Gemini extraction (explicitly timeboxed)
- Reliability: System must work even if MongoDB is down (in-memory fallback = NFR enforcement)
- Usability: Non-technical users must complete the workflow without technical knowledge
- Security: API keys must never be exposed to clients

The SRS document in documentation/SRS.pdf is the formal specification artifact. The Zod schemas and TypeScript types in the codebase provide implementation-level traceability back to that specification.

7. Software Process

The project follows Incremental Agile development:

- 75+ commits demonstrate continuous delivery of working software
- Templates added one at a time — each release adds new functionality incrementally

- Gemini integration, then MongoDB, then QR codes — features built and integrated iteratively
- CLAUDE.md and AGENTS.md in portfolio-templates suggest AI-assisted development within an agile workflow

The high uncertainty around AI output quality (Gemini parsing accuracy) also aligns with the Spiral Model — each sprint involved risk analysis (does Gemini handle non-tech CVs?), prototyping (try it), then committing the implementation.

Summary Table — SE Concepts in ResuFlow

SE Concept	Where in ResuFlow	Why
Layering	Entire 3-tier system (Frontend → Backend → DB)	Separation, substitutability
Client-Server	React/Next.js ↔ Express API	Core communication model
Pipe & Filter	fileParser → cvExtractor → schemaValidator	Data transformation chain
Repository	MongoDB + in-memory Map	Shared data across 3 subsystems
Modularity	routes/ services/ models/ components/	Isolate changes
Information Hiding	cvExtractor hides Gemini, fileParser hides PDF/DOCX	Swap implementations freely
Interfaces	REST API, TypeScript types, service function signatures	Decouple subsystems
Abstraction	AppError, fileParser, config object	Simplify caller code
Generality	Multi-format parser, multi-profession CV schema	Reuse across contexts
Cohesion/Coupling	Services do one thing; subsystems don't import each other	Safe evolution
Design by Contract	Zod, Multer config, schemaValidator, rate limiter	Enforce boundaries
CBSE	React components, Radix UI, template folders	Composable, replaceable units
Modifiability	Add template = one new folder, no core changes	Long-term maintainability
Reliability	Retry logic, timeouts, MongoDB fallback	Survive external failures
Robustness	Backend re-validates frontend data	Mutual suspicion principle
Security	File validation, .env, CORS, rate limiting, Zod	Protect attack surfaces
UI Golden Rules	Breadcrumbs, localStorage, consistent Navbar, Radix	Usable for non-tech users
Incremental Agile	Templates added one-by-one, 75+ commits	Respond to change